TECNOLOGÍAS DE LA INFORMACIÓN

Desarrollo de Sistemas Informáticos

4 créditos

Profesores:

Ing. Roly Steeven Cedeño Menéndez, Mtr.

Titulaciones	Semestre
• *Tecnologías de la Información en Línea*	*Quinto*

NOMBRES Y APELLIDOS: Jeniffer Lilibeth Cedeño Andrade

PARALELO: "A"

SEMESTRE: Quinto Semestre

ACTIVIDAD: Actividad #1: Sistema de Control de Rescate y Adopción Animal

Es calibri (cuerpo) tamaño de letra 11

PERIODO SEPTIEMBRE 2025 - ENERO 2026

Actividades a desarrollar en la Unidad 1

Actividad # 1

Introducción

Para el rescate y una adopción eficaz, se necesita tener un adecuado sistema de organización para que el seguimiento de los casos sea de fácil acceso, también asignar responsables como de igual manera la intervención de los procesos que se llevan a cabo para la recuperación de los animales. El diseño de un Sistema Informático óptimo ayuda a contribuir a la organización de los registros y mejoración del manejo de las actividades que se realizan por los ciudadanos, voluntarios veterinarios y el personal de administración del centro de refugios de los animales.

Para desarrollar un Sistema Informático es indispensable concretar una arquitectura lógica que permita organizar los mecanismos del sistema y establecer como se relacionan entre sí. Según Object Management Group (2017), el Lenguaje Unificado de Modelado (UML) brinda herramientas para representar y documentar la estructura y el comportamiento de un sistema mediante diagramas. También se empleará principios de diseño SOLID y patrones de diseño que permita separar responsabilidades y facilitar futuras modificaciones. Al utilizar GitHub con el flujo de trabajo Gitflow, se registran los avances y se mantiene la trazabilidad de las versiones de diseño.

En el presente estudio de caso a desarrollar, plantea un Sistema Informático de Control de Rescate y Adopción Animal, que permitirá el registro de animales rescatados, clasificarlos según su especie, delegar una persona encargada, y actualizar sus estados, teniendo en cuenta que el seguimiento será desde el momento que se rescata el animal hasta la adopción, a continuación, en la figura 1 se muestra un ejemplo de la trazabilidad de un animal.

Figura 1

La trazabilidad de un animal



Nota: Imagen tomada de Google, *que rastrea el Origen, el Proceso (la veterinaria/refugio) y el Destino (el hogar definitivo).*

2. OBJETIVOS

2.1. Objetivo general

Diseñar la arquitectura lógica de un Sistema de Control de Rescate y Adopción Animal mediante el análisis de requerimientos, el modelado UML, la aplicación de patrones de diseño y el uso de Gitflow, con el propósito de plantear una solución organizada, escalable y mantenible.

2.2. Objetivos específicos

- Identificar los actores (Ciudadano/Reportante, Voluntario/Veterinario, Administrador del Refugio) que interactúan con el sistema y establecer los requerimientos funcionales y no funcionales.
- Representar la estructura y el comportamiento del sistema a través de diagramas UML de casos de uso, clases y secuencias
- Aplicar y justificar patrones de diseño adecuados para la creación de expedientes, la gestión centralizada de casos y las notificaciones automáticas.
- Definir una arquitectura lógica que separe las responsabilidades de la interfaz, la lógica de negocio y el acceso a los datos.
- Documentar los avances del diseño mediante un repositorio de GitHub organizado con el flujo de trabajo Gitflow.

3. ALCANCE DEL SISTEMA

El sistema está enfocado en la administración de los casos de rescate que son atendidos por un refugio para animales. Incluirá el registro de los animales, la determinación de su especie, la asignación de veterinarios o voluntarios y el monitoreo de su condición. Los estados que se tendrán en cuenta para el proceso son: Rescatado, en terapia, apto para adopción y adoptado. Las circunstancias del caso y las acciones que los encargados del refugio han autorizado determinarán la modificación de estado.

La plataforma incluirá tres categorías principales de usuarios: los ciudadanos o reportantes, los veterinarios o voluntarios y los encargados del refugio. Cada actor tendrá responsabilidades particulares de acuerdo con sus funciones.

4. IDENTIFICACIÓN DE ACTORES DEL SISTEMA

Los actores que se mencionan son las personas que interactúan con el Sistema de Control de Rescate y Adopción animal. A continuación, en la tabla 1 se identifican los tres actores en base al escenario de la actividad.

Tabla 1

Identificación de los autores en el Sistema Informático

Actor	Descripción / rol	Acciones Principales en el Sistema
Ciudadano/Reportante	Usuario externo (público general) que interactúa con la plataforma para alertar sobre animales en riesgo.	• Registrarse en la plataforma. • Crear caso de rescate (reportar ingreso). • Subir fotos y ubicación del animal • Consultar estado de su reporte.
Voluntario/Veterinario	Personal operativo técnico e interno del refugio, encargado del cuidado	• Ver casos de rescate asignados. • Actualizar estado del proceso (rescatado, Tratamiento, Adopción). • Registrar historial médico y tratamientos.

	directo y seguimiento de los animales.	• Recibir notificaciones de nuevos casos.
Administrador del Refugio	Usuario con los permisos más altos en la plataforma. Tiene el control total para supervisar y gestionar el funcionamiento del refugio.	• Gestionar usuarios (Voluntarios/Veterinarios). • Asignar personal encargado a cada caso. • Categorizar especies (Perro, Gato, Fauna Silvestre). • Aprobar solicitudes de adopción.

Nota: Elaboración propia

5. REQUERIMIENTOS DEL SISTEMA

5.1. Requerimientos funcionales

Son las acciones que el sistema debe permitir realizar, ver tabla 2.

Tabla 2

Acciones que el sistema permite

Código	Requerimiento funcional
RF01	El sistema permitirá registrar casos de animales rescatados.
RF02	El sistema permitirá registrar los datos del animal, como identificación, especie y descripción.
RF03	El sistema permitirá clasificar los animales en perros, gatos o fauna silvestre.
RF04	El sistema permitirá asignar un voluntario o veterinario responsable a cada caso.
RF05	El sistema permitirá consultar los casos registrados y sus responsables.
RF06	El sistema permitirá actualizar el estado del caso, por ejemplo: rescatado, en tratamiento o en proceso de adopción.
RF07	El sistema permitirá notificar a los voluntarios correspondientes cuando se registre un nuevo caso de rescate.
RF08	El sistema permitirá registrar la información relacionada con la adopción de un animal.

Nota: Elaboración propia

5.2. Requerimientos no funcionales

Son las características de calidad que debe considerar el diseño del sistema como se muestra en la tabla 3.

Tabla 3

Características del diseño del sistema

Código	Requerimiento no funcional
RNF01	**Mantenibilidad:** el diseño deberá facilitar la modificación y actualización de sus componentes.
RNF02	**Escalabilidad:** la arquitectura deberá permitir incorporar nuevas funcionalidades y tipos de animales.
RNF03	**Trazabilidad:** los cambios del diseño deberán registrarse mediante Git y GitHub.
RNF04	**Seguridad:** el diseño deberá considerar permisos de acceso según el rol de cada usuario.
RNF05	**Organización:** las clases y componentes deberán tener responsabilidades claramente definidas.

Nota: Elaboración propia

EVIDENCIAS

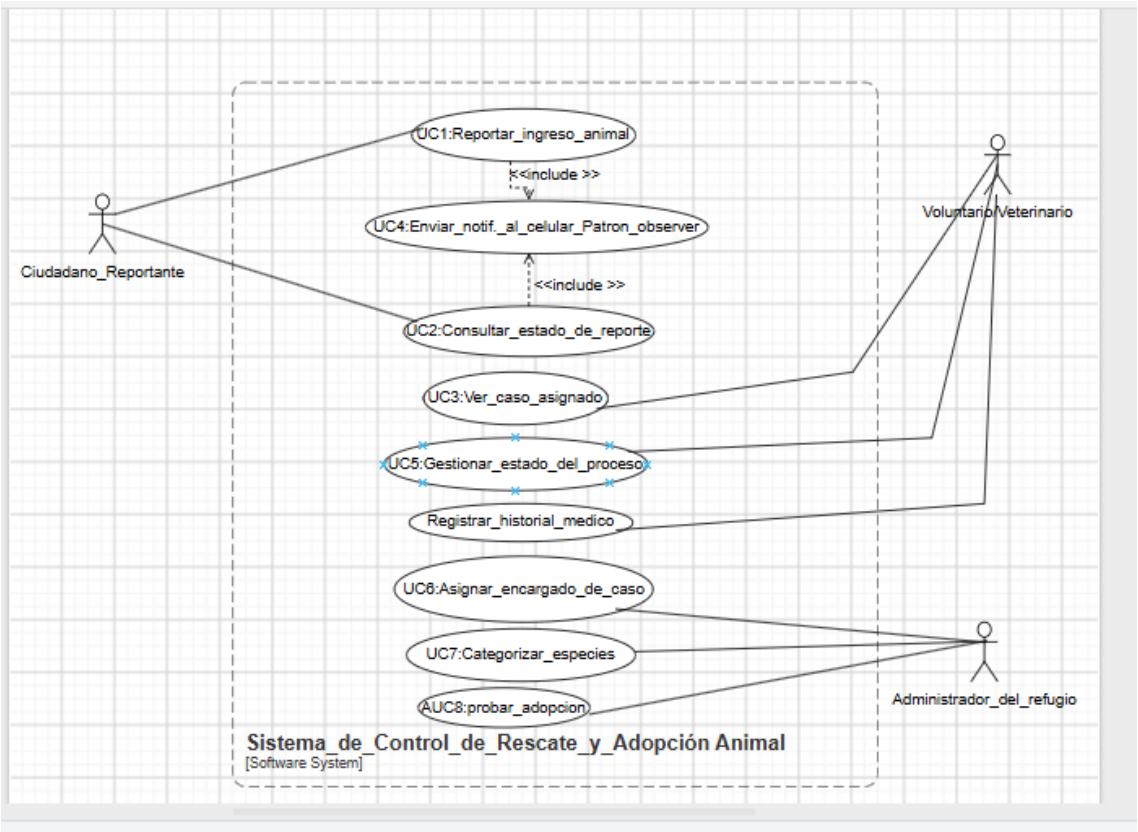
(*Código de la ejecución de los ejercicios, enlace drive donde se encuentren los mismos y Capturas de pantalla claras y relevantes, donde evidencie el proceso realizado).*

6. DIAGRAMA **UML** DE CASOS DE USO

Este diagrama muestra quién manipula el sistema y qué acciones puede realizar cada actor. En la figura 1 se muestra el diagrama de caso de cada autor.

Figura 1

Acciones de cada autor

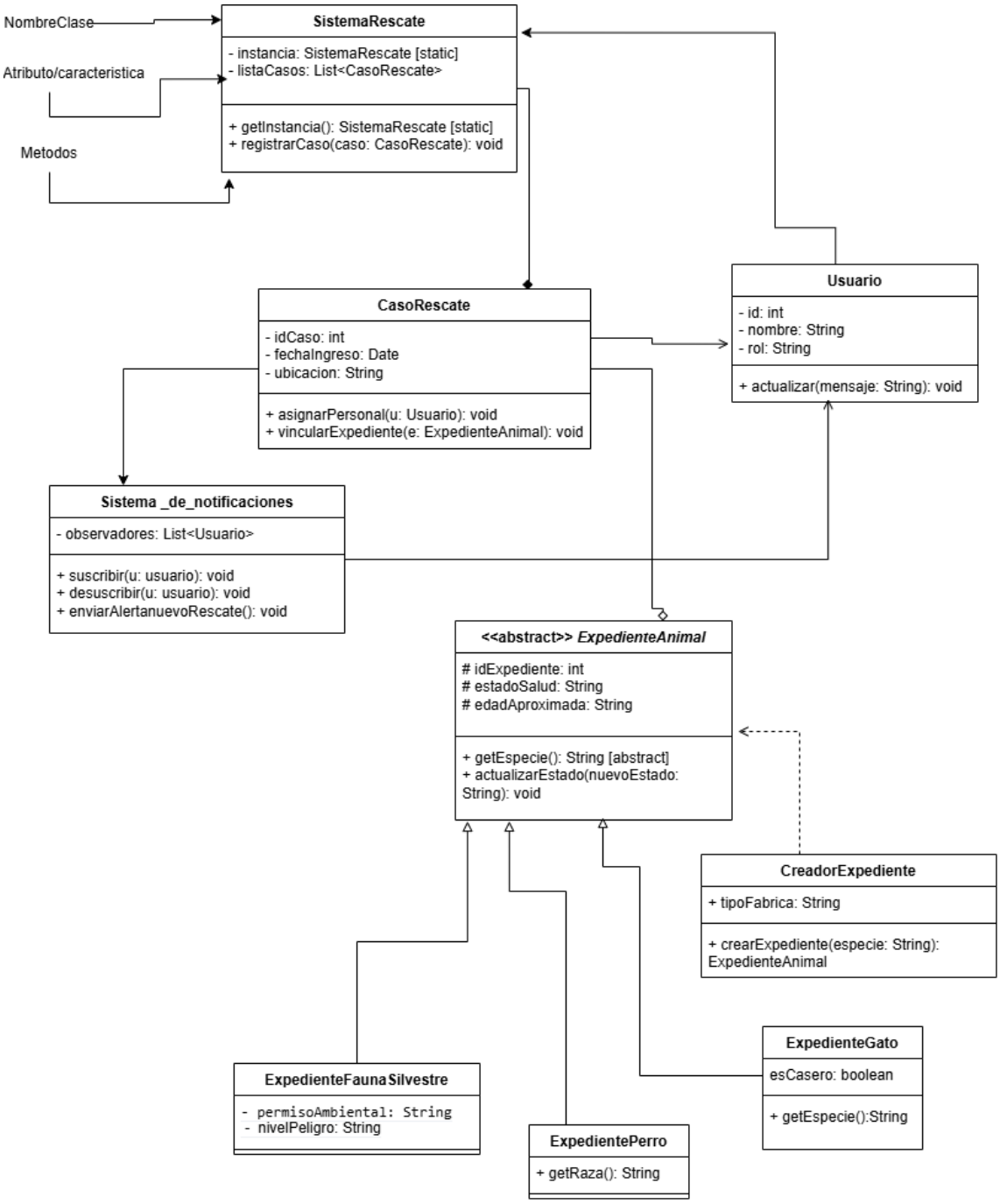


Nota: Elaboración propia hecho, en el Software Draw.io - https://app.diagrams.net/.

Paso 7. DIAGRAMA DE CLASES EN Diagrams.net

Figura 2

Diagrama de clases



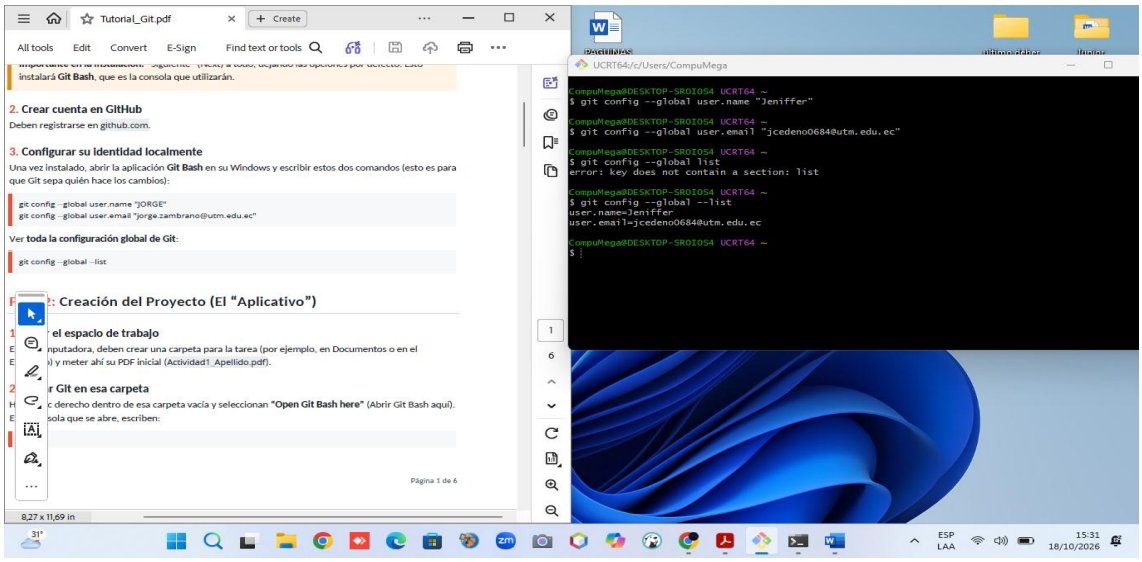
***Nota**: Elaboración propia, hecho en el Software Draw.io - https://app.diagrams.net/.*

Consola Git Bash con los comandos iniciales

En la figura 3 se observa la configuración por primera vez de la aplicación **Git Bash** en el Windows para que Git sepa quién hace los cambios.

Figura 3

Comandos iniciales

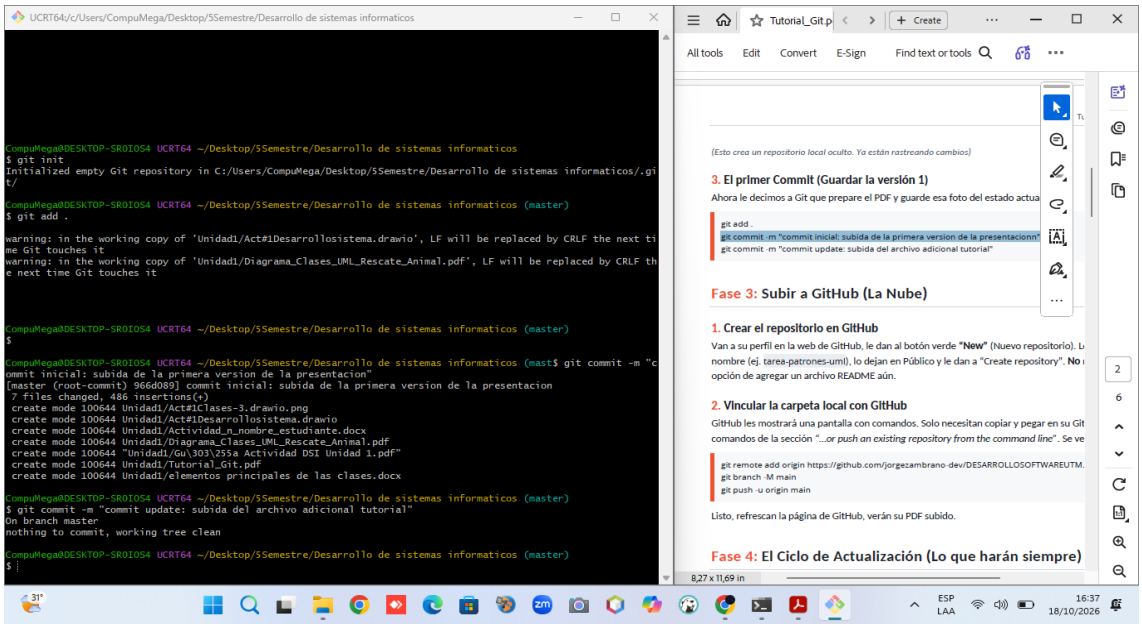


***Nota**: Elaboración propia*

En la figura 4 se muestra la aparición de la primera versión, en la cual muestra la carpeta que contiene algunos archivos Word y PDF.

Figura 4

Primera versión

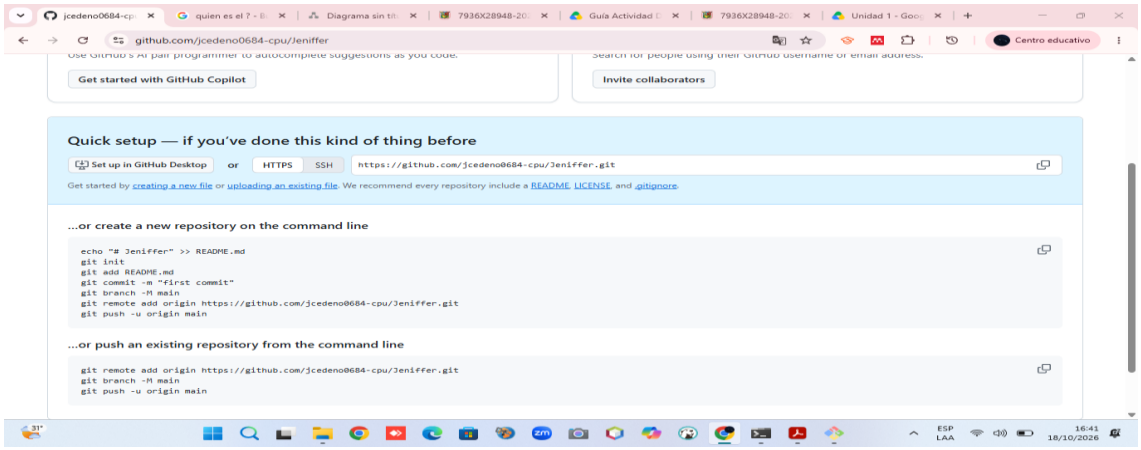


***Nota**: Elaboración propia*

Aquí se entró en la github.com

Figura 5

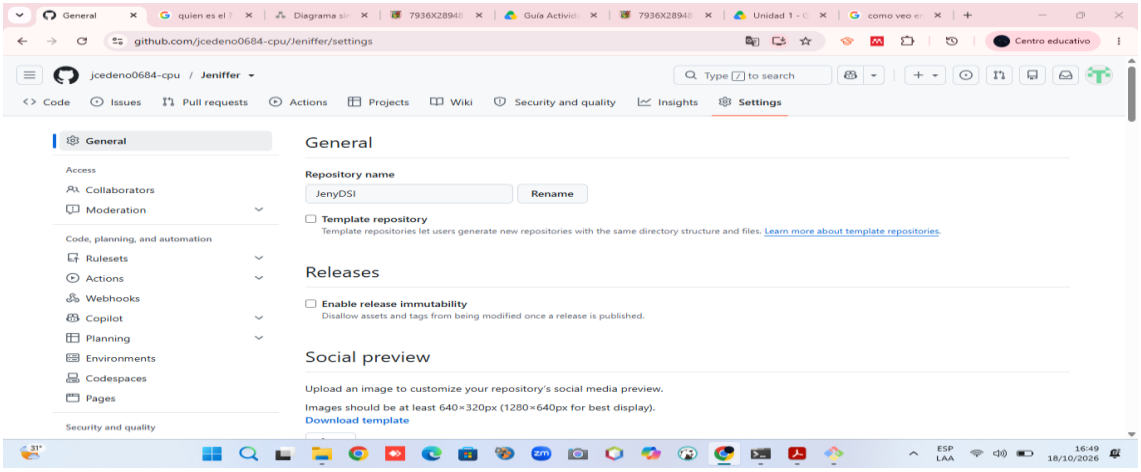
Inicio de sesión github.com



***Nota**: Elaboración propia*

Figura 6

Repositorio renombrado a JenyDSI

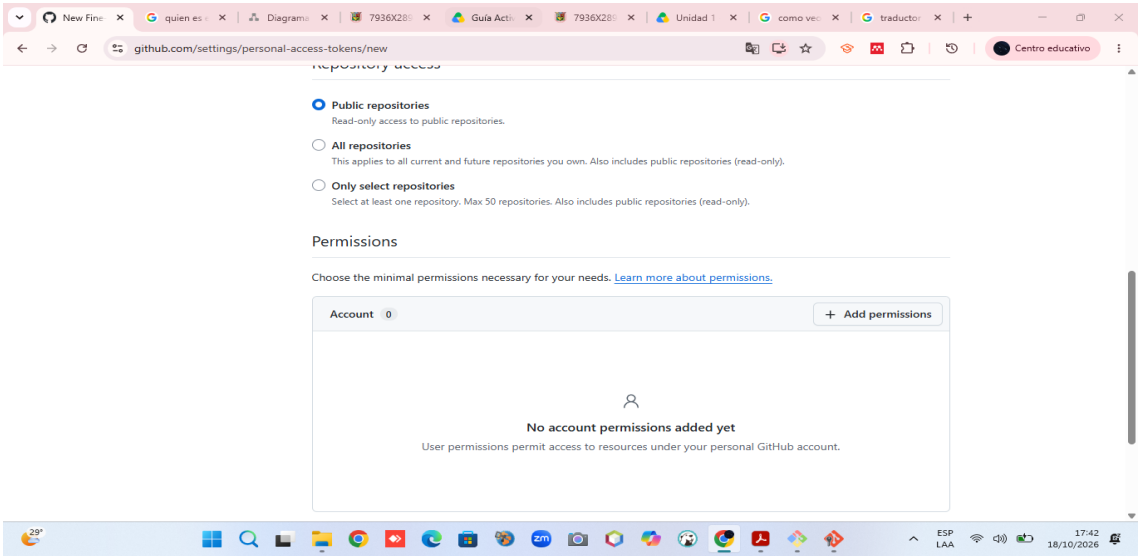


***Nota**: Elaboración propia*

¡A continuación de la instalación se presentaron unos errores en consola, pero fue porque no estaba habilitado ciertos permisos en la parte del Token, arreglándose con éxito!

Figura 7

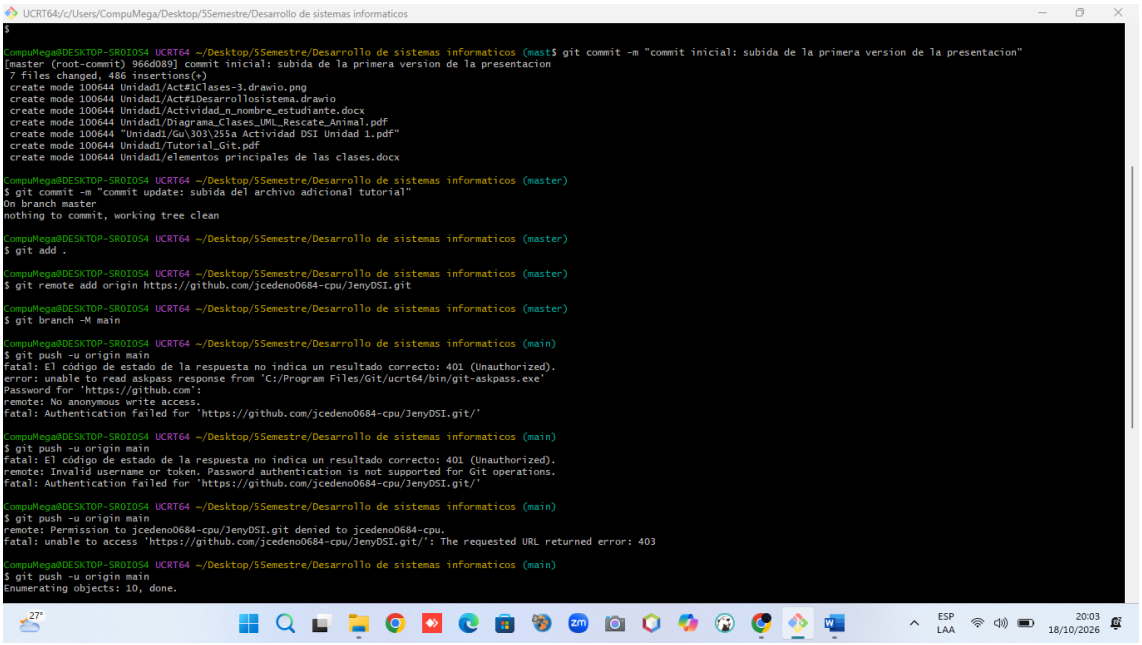
Ventana de permisos en Fine-grained tokens



***Nota**: Elaboración propia*

Figura 8

Errores que se dieron en: la conexión a GitHub

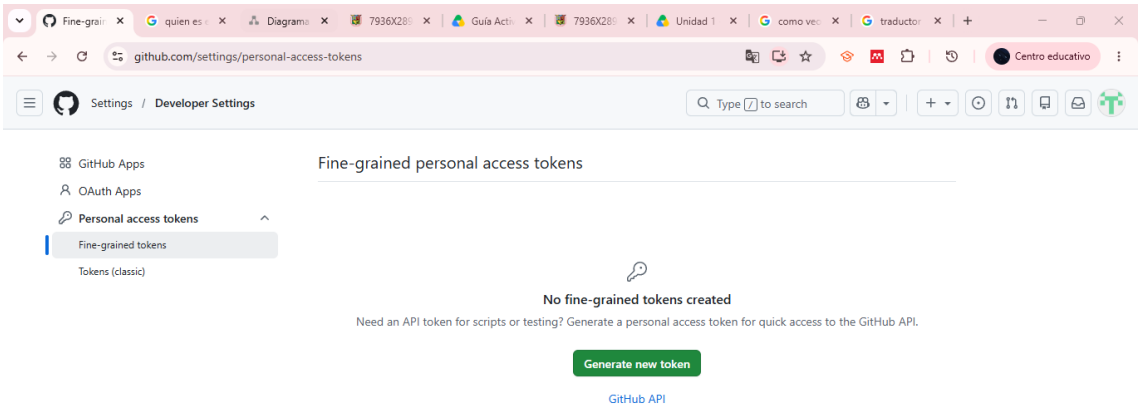


***Nota**: Elaboración propia*

Se solucionó seleccionando en el token clásico y activando los permisos necesarios.

Figura 10

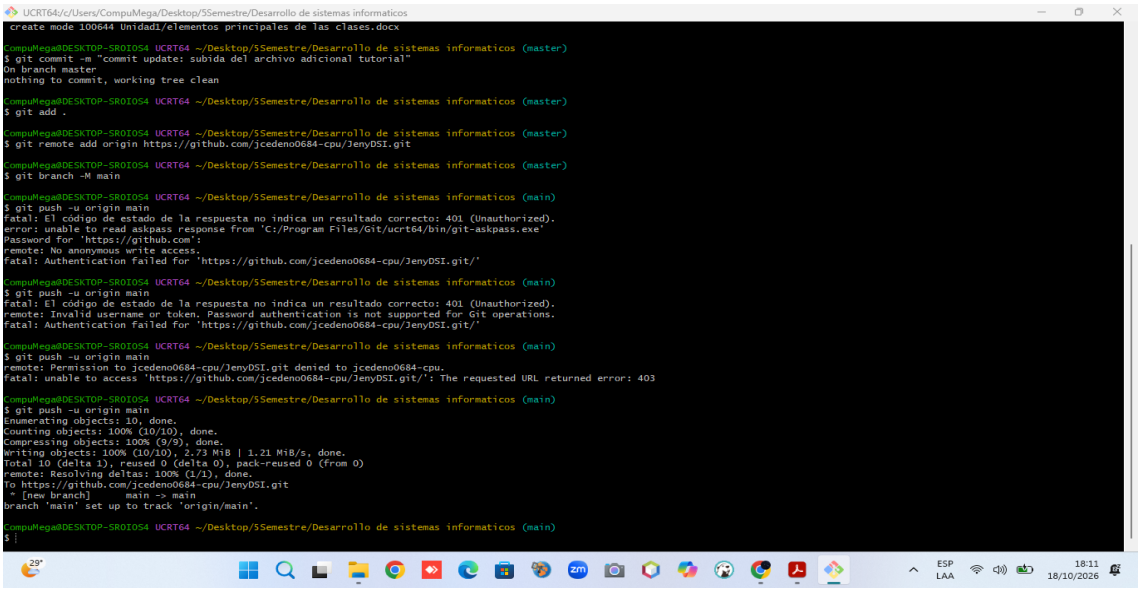
Click en Tokens (classic)



***Nota**: Elaboración propia*

Figura 10

Conexión exitosa

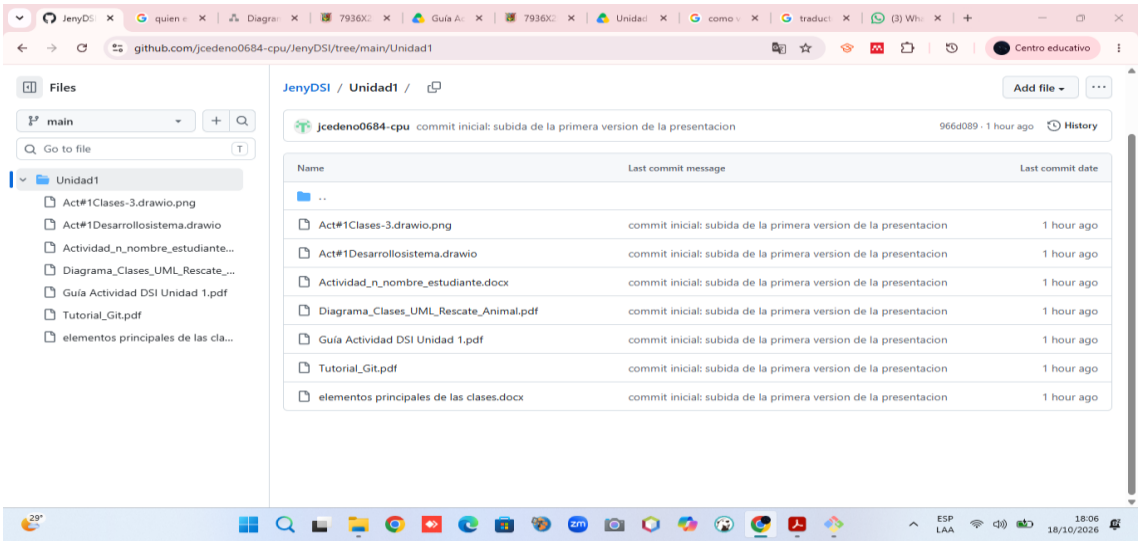


***Nota**: Elaboración propia*

Una vez enlazado con éxito se pudo *continuar* con la configuración, ver figura 11 .

Figura 11

Repositorio actualizado

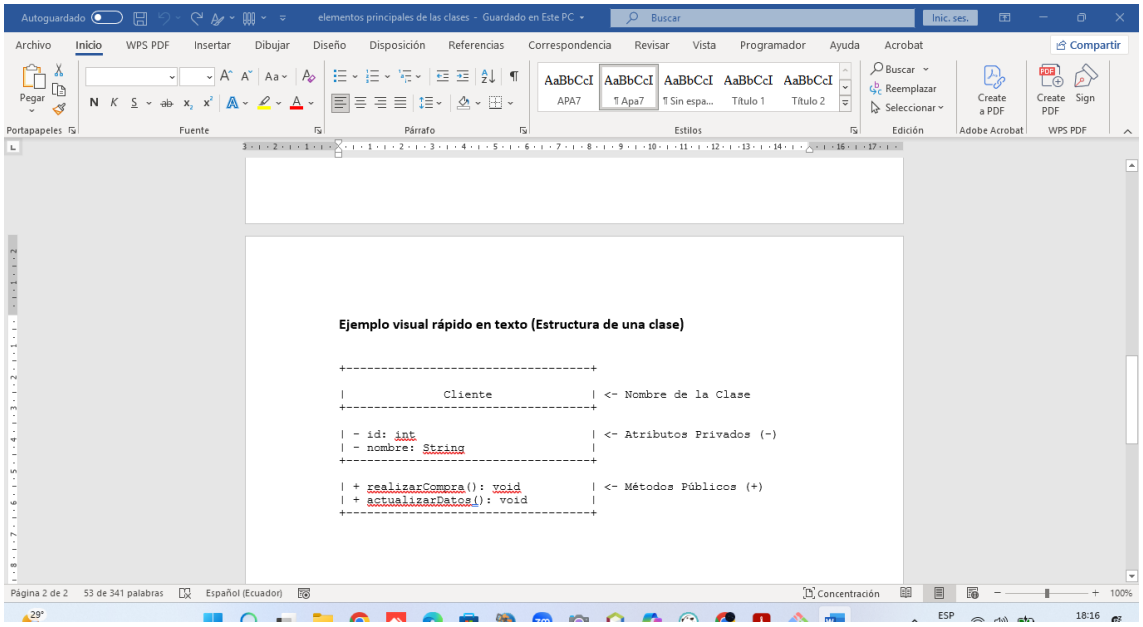


***Nota**: Elaboración propia*

En la figura 12 se observa el segundo cambio donde se realizo en un documento de Word borrando la ultima parte archivo guardado.

Figura 12

Documento de Word como prueba para el segundo cambio

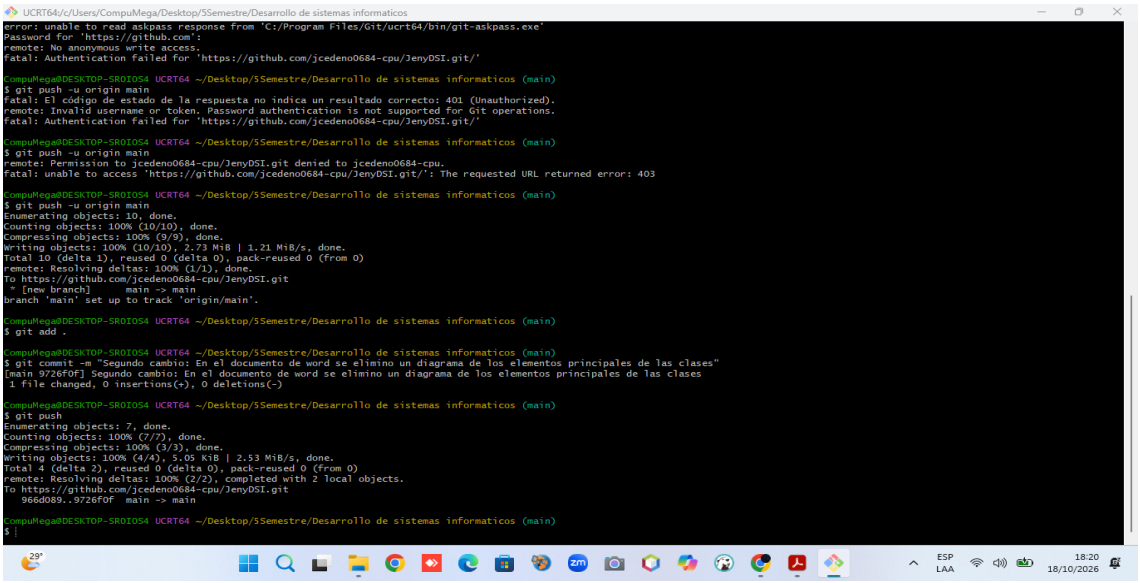


***Nota**: Elaboración propia*

En la figura 13 se ingresó los comandos que se observan para registrar o actualizar los cambios.

Figura 13

Cambios realizados

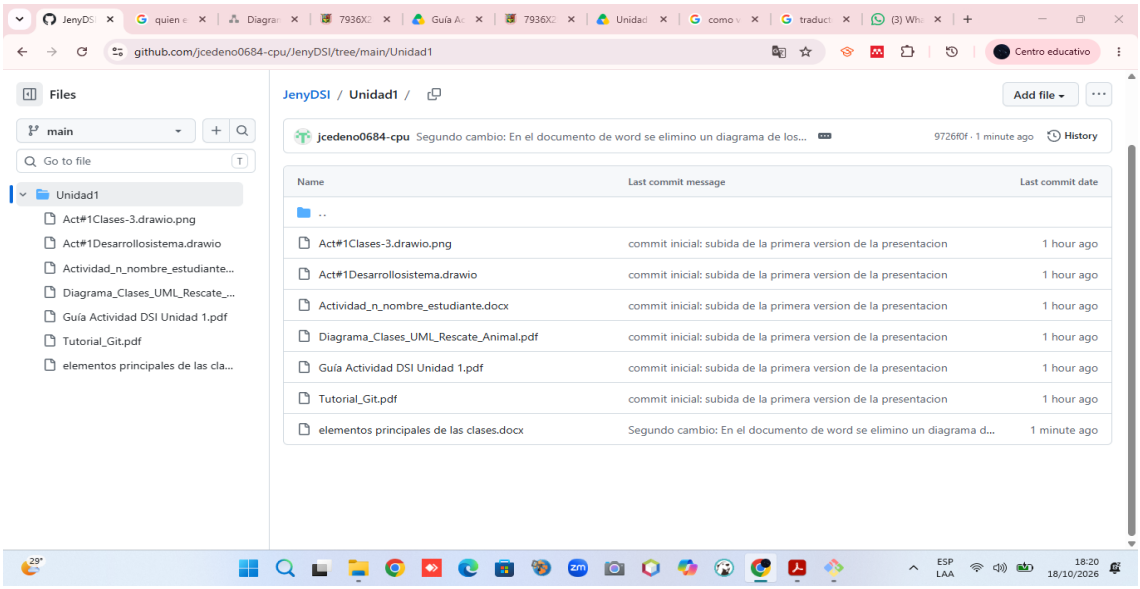


***Nota**: Elaboración propia*

En la figura 14 ya se observa el segundo cambio hecho en consola

Figura 14

Segundo cambio



***Nota**: Elaboración propia*

Bibliografía

Object Management Group. (2017). OMG Unified Modeling Language (OMG UML), version 2.5.1. https://www.omg.org/spec/UML/2.5.1

Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design patterns: Elements of reusable object-oriented software. Addison-Wesley Professional.

Martin, R. C. (2002). Agile software development: Principles, patterns, and practices. Prentice Hall.

Atlassian. (s. f.). Flujo de trabajo de Gitflow. https://www.atlassian.com/es/git/tutorials/comparing-workflows/gitflow-workflow

Refactoring.Guru. (s. f.). Singleton. https://refactoring.guru/es/design-patterns/singleton

Refactoring.Guru. (s. f.). Observer. https://refactoring.guru/es/design-patterns/observer

Refactoring.Guru. (s. f.). Factory Method. https://refactoring.guru/es/design-patterns/factory-method

Enlaces de los diagramas hecho en github.com

https://app.diagrams.net/#G1Bw0TrWxLkLRZ6nVfjErkTpPGn8Kv5lSz#%7B%22pageId%22%3A%2252opxhc2MYbXhIZrIKbe%22%7D

Enlace de GitHub

https://github.com/jcedeno0684-cpu/JenyDSI